# CHAPTER FIVE
# GEOMETRIC TRANSFORMATION

# OUTLINE

- **Introduction**
- **Matrix Representation**
- **2-D Transformations**
- **Homogeneous Coordinates**
- **3-D Transformations**
- **Combination of Transformations**
- **Transformation in OpenGl**

# INTRODUCTION

- Operations that are applied to the geometric description of an object to change its position, orientation, or size are called geometric transformations.
- Transformations play a very important role in manipulating objects on the screen. The modified object is displayed without having to redraw it.
- Transformations in OpenGL are not drawing commands. They are retained as part of the graphics state.
- **Types of Transformations**
  - Translation (movement)
  - Rotation (turning)
  - Scaling (resizing)

# BASICS OF MATRICES

- A matrix is an array of numbers.
- Matrices can be added component wise, provided they have the same dimensions:

$$\begin{bmatrix} a11 & a12 \\ a21 & a22 \end{bmatrix} + \begin{bmatrix} b11 & b12 \\ b21 & b22 \end{bmatrix} = \begin{bmatrix} a11+b11 & a12+b12 \\ a21+b21 & a22+b22 \end{bmatrix}$$

- Matrix A (m × n) can multiply Matrix B (n × p) → Result is (m × p)

$$\begin{bmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \end{bmatrix} * \begin{bmatrix} b11 & b12 \\ b21 & b22 \\ b31 & b32 \end{bmatrix} =$$

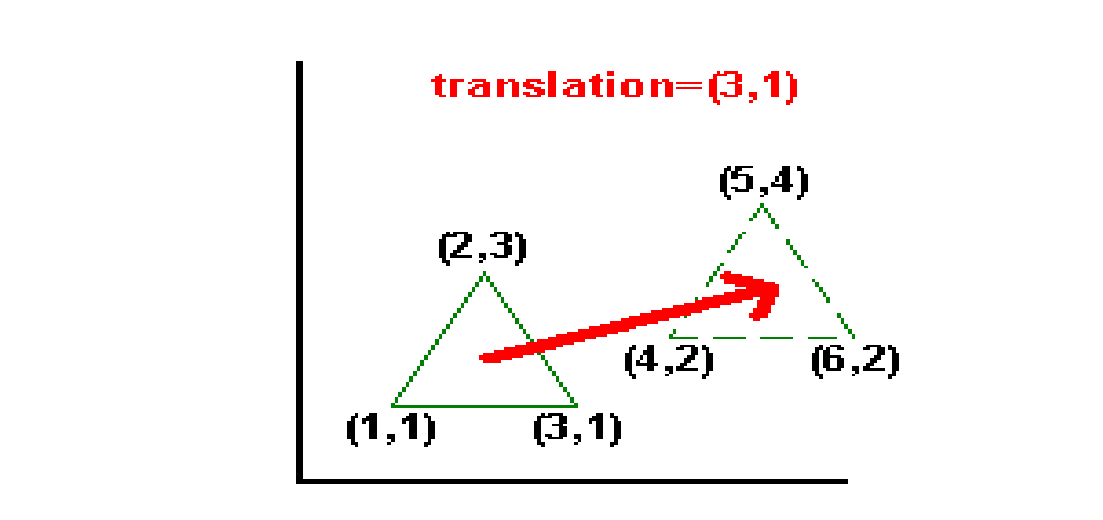
$$\begin{bmatrix} a11*b11 + a12*b21 + a12*b31 & a11*b12 + a12*b22 + a13*b32 \\ a21*b11 + a22*b21 + a23*b31 & a21*b12 + a22*b22 + a23*b32 \end{bmatrix}$$

# 2-D Translation

- Translation is the transform applied when we wish to move an object. It **shifts** all points by the same **amount**.
- To move a point P by a distance (tx,ty), all points of P(x,y) on this shape will move to a new location P'(x,y) = P(x+tx, y+ty).
- To translate a point *P* to *P'* we add on a vector *T*:-
- P = $\begin{bmatrix} x \\ y \end{bmatrix}$, and translation vector as T= $\begin{bmatrix} Tx \\ Ty \end{bmatrix}$
- Transformed point P‘ is the vector represented as P’ = $\begin{bmatrix} x + Tx \\ y + Ty \end{bmatrix}$

# CONT.

- Let's take a point: P=(2,3), translate by: T=(3,1), => then: P′=(2+3,3+1)=(5,4)
- Translate P(4,2) by T(-1,3): P' = (4-1, 2+3) = (3,5)
- Example:-the following Figure show that translation of triangle by translation vector (3,1).




**Exercise**

1. Translate (1,1) by (2,3)
2. Translate (5,4) by (-2,-1)

# 2-D Rotation

- A shape can be rotated about any axes.
- The rotation transformation rotates all points about a **centre of rotation**.
- This centre of rotation is assumed to be at the origin (0,0), but it is possible to rotate about any point.
- The rotation transformation has a single parameter: the angle of rotation, $\theta$.
- To rotate a point $P$ anti-clockwise **by $\theta^o$**, we apply the rotation matrix $R$:

$$R = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \quad \text{.....} \quad (8)$$

$$\begin{pmatrix} p'_x \\ p'_y \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} p_x \\ p_y \end{pmatrix} \quad \text{.....} \quad (9)$$

# CONT.

- Formula

  $x' = x \cos\theta - y \sin\theta$
  $y' = x \sin\theta + y \cos\theta$

- Example (90° Rotation): Rotate (1,0) by 90°: -

  $x' = 1*\cos(90°) - 0 \sin(90°) = 0 + 0$
  $y' = 1*\sin(90°) + 0 \cos(90°) = 1 + 0$

  $\Rightarrow P' = (0,1)$

- Exercise: Rotate (2,0) by 180°

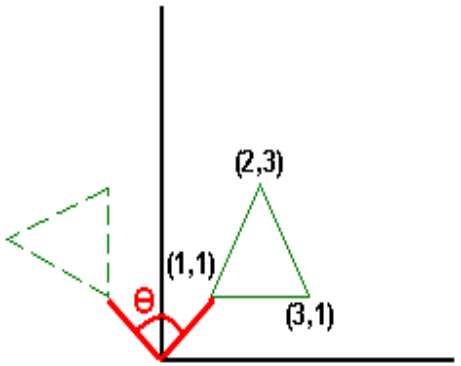



Figure 2 - A 2-D Rotation about the Origin

# 2-D Scaling

- ✓ The scaling transformation multiplies each coordinate of each point by a scale factor to changes the size of an object.
- ✓ The scale factor can be different for each coordinate.
- ✓ If all scale factors are equal, we call it uniform scaling, if they are different, we call it anamorphic or differential scaling.
- ✓ To scale a point P by scale factors $S_x$ and $S_y$ we apply the scaling matrix S:

$$S = \begin{pmatrix} S_x & 0 \\ 0 & S_y \end{pmatrix} \quad \text{.....} \quad (12)$$

$$\begin{pmatrix} p'_x \\ p'_y \end{pmatrix} = \begin{pmatrix} S_x & 0 \\ 0 & S_y \end{pmatrix} \begin{pmatrix} p_x \\ p_y \end{pmatrix} \quad \text{.....} \quad (13)$$

# CONT.

- **Formula**

  $P(x, y) \rightarrow P'(Sx \cdot x, Sy \cdot y)$

- **Example 1**

  Scale (2,3) by (2,2): P' = (4,6)

- **Example 2**

  Scale (3,2) by (1,3): P' = (3,6)

- The following figure show 2D scaling with scale factor (2,2).

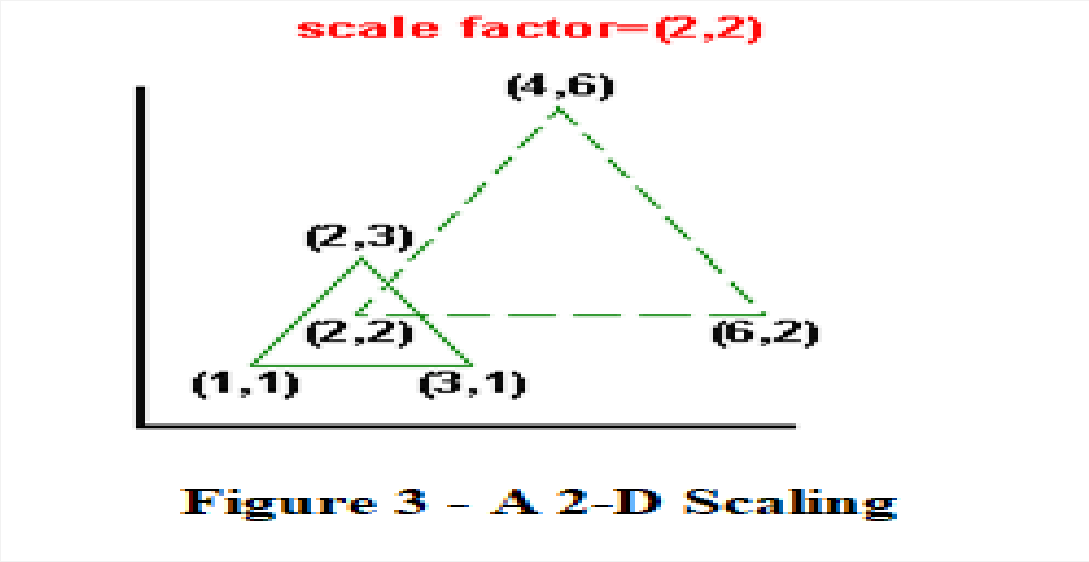



Figure 3 – A 2-D Scaling

# HOMOGENEOUS COORDINATE

- A sequence of transformations that every primitive undergoes before being displayed is called pipeline.
- As we must perform a sequence of transformations in this pipeline, it is essential that we have an efficient way to execute these transformations.
- Unfortunately these all differ in their representations and cannot be combined in a consistent manner.
- To treat all transformations in a consistent way, the concept of homogeneous coordinates was applied to CG.
- With homogeneous coordinates we add an extra coordinate, the *homogeneous parameter*, to each point in Cartesian coordinates.
- Allows all transformations (including translation) to use matrix multiplication

# CONT.

- The relationship between homogeneous points and their corresponding Cartesian points is:-

$$\text{Homogeneous point} = \begin{pmatrix} x \\ y \\ h \end{pmatrix}, \text{Cartesian point} = \begin{pmatrix} x/h \\ y/h \\ 1 \end{pmatrix}$$

- **Conversion:**
  - **To Homogeneous:** Add a 1 as the last coordinate (e.g., (x,y -> (x,y,1)).
  - **To Cartesian:** Divide all coordinates by the last coordinate (h).
  - If h=0, it represents a point at infinity (direction vector)

# CONT.

- Homogeneous parameter helps us to express translation transformations using matrix multiplications.
- This allows efficient concatenation of transformations (e.g., rotation, then translation) into a single matrix.
- Since matrix multiplications are often executed in dedicated hardware on the video card this will be very fast.

# 2-D Translation with Homogeneous Coordinates

- Now we can express a translation transformation using a single matrix multiplication, as shown below.

$$P = \begin{pmatrix} p_x \\ p_y \\ 1 \end{pmatrix} \quad \text{.....} \quad (16)$$

$$P' = \begin{pmatrix} p'_x \\ p'_y \\ 1 \end{pmatrix} \quad \text{.....} \quad (17)$$

$$T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \quad \text{.....} \quad (18)$$

$$\begin{pmatrix} p'_x \\ p'_y \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ 1 \end{pmatrix} \quad \text{.....} \quad (19)$$

Therefore $p'_x = p_x + t_x$, $p'_y = p_y + t_y$, exactly the same as before, but we used a matrix multiplication instead of an addition.

# 2-D Rotation with Homogeneous Coordinates

- Rotations can also be expressed using homogeneous coordinates.
- The following equations are similar to the form of the 2-D rotation given above.

$$R = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \quad (20)$$

$$\begin{pmatrix} p'_x \\ p'_y \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ 1 \end{pmatrix} \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \quad (21)$$

$$p'_x = p_x \cos\theta - p_y \sin\theta \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \quad (10)$$

$$p'_y = p_y \cos\theta + p_x \sin\theta \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \quad (11)$$

# 2-D Scaling with Homogeneous Coordinates

- Finally, we can also express scaling using homogeneous coordinates, as shown by the following equations.

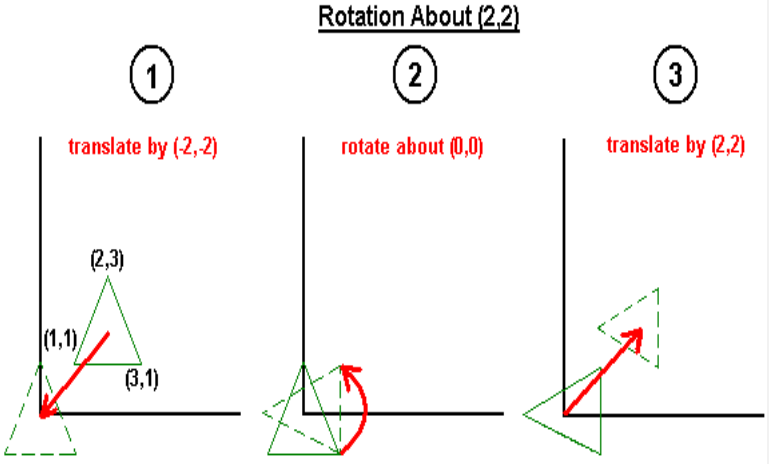
$$S = \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \quad (22)$$

$$\begin{pmatrix} p'_x \\ p'_y \\ 1 \end{pmatrix} = \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ 1 \end{pmatrix} \quad \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \quad (23)$$

Therefore $p'_x = S_x p_x$ and $p'_y = S_y p_y$, exactly the same as before.

# ROTATION AT PIVOT POINT

- The rotation matrix we introduced in previous section only rotates about the origin, but often we may want to apply a rotation about a different point (a *pivot point*).
- we can achieve this using the following sequence of transformations:-
  - Translate from pivot point to origin
  - Rotate about origin
  - Translate from origin back to pivot point




- Exercise:- Rotate the point P(5,5) by **90° counter clockwise** about the pivot point **C(2,3)**.

# COMBINATION OF TRANSFORMATIONS

- Transformations can be combined. One important rule is Order matters!
  Scale then Translate ≠ Translate then Scale
- Important:- scaling after translation also scales the translation distance.
- For each object, the usual sequence is: Translate, Rotate then Scale
- Example: Apply scaling (2,2) then translation (1,1) to point (1,1)

  Step 1: Scaling (2, 2) => Multiply the original coordinates by the scaling factors

  x' =1*2=2, y'=1*2 =2

  Intermediate Point: (2, 2)

  Step 2: Translation (1, 1) => Add the translation values to the intermediate coordinates.

  x' =2+1=3, y'=2+1 =3

  => Final Point: (3, 3)
- Exercise

  Reverse the order and compare results

# 3-D MATRIX TRANSFORMATIONS

- In 3D, points are (x, y, z)
- **Types**
  - Translation (tx, ty, tz)
  - Scaling (Sx, Sy, Sz)
  - Rotation about x, y, z axes

## Translation

We can see that 3-D translations are defined by three translation parameters: $t_x$, $t_y$ and $t_z$. We apply this transformation as follows:

$$\begin{pmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix} \quad \text{.....} \quad (26)$$

Therefore $p'_x = p_x + t_x$, $p'_y = p_y + t_y$ and $p'_z = p_z + t_z$.

# SCALING

Similarly, 3-D scalings are defined by three scaling parameters, $S_x$, $S_y$ and $S_z$. The matrix is:

$$S = \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

We apply this transformation as follows:

$$\begin{pmatrix} p'_x \\ p'_y \\ p'_z \\ 1 \end{pmatrix} = \begin{pmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} p_x \\ p_y \\ p_z \\ 1 \end{pmatrix}$$

Therefore $p'_x = S_x p_x$, $p'_y = S_y p_y$ and $p'_z = S_z p_z$.

# 3-D ROTATION

- For rotations in 3-D we have three possible *axes of rotation*: the *x*, *y* and *z* axes. (Actually we can rotate about any axis, but the matrices are simpler for *x*, *y* and *z* axes.)
- Therefore the form of the rotation matrix depends on which type of rotation we want to perform.

For a rotation about the $x$-axis the matrix is:

$$R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

For a rotation about the $y$-axis we use:

$$R_y = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

And for a rotation about the $z$-axis we have:

$$R_z = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

# CONT.

- OpenGL functions for defining transformations.
  - ✓ glTranslatef($t_x$,$t_y$,$t_z$): multiply the current matrix by a translation matrix formed from the translation parameters $t_x$, $t_y$ and $t_z$.
  - ✓ glRotatef(θ,vx,vy,vz): multiply the current matrix by a rotation matrix that rotates by $\theta^o$ about the axis defined by the direction of the vector (vx,vy,vz).
  - ✓ glScalef($S_x$,$S_y$,$S_z$): multiply the current matrix by a scaling matrix formed from the scale factors $S_x$, $S_y$ and $S_z$.
- Each current matrix in OpenGL has an associated matrix stack LIFO.
- Use glPushMatrix() to save the current matrix and glPopMatrix() to restore it.
- This is useful for hierarchical transformations (e.g., robot arms, solar systems).

# OPENGL CODE EXAMPLE

```
void display() {
   glClear(GL_COLOR_BUFFER_BIT);
   glLoadIdentity();

   glScalef(2.0f, 2.0f, 1.0f);

   glBegin(GL_TRIANGLES);
      glVertex2f(0.0f, 0.0f);
      glVertex2f(1.0f, 0.0f);
      glVertex2f(0.5f, 1.0f);
   glEnd();

   glFlush();
}
```

```
void display() {
glClear(GL_COLOR_BUFFER_BIT);
glLoadIdentity();

glTranslatef(1.0f, 1.0f, 0.0f);
glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
glScalef(1.5f, 1.5f, 1.0f);

glBegin(GL_TRIANGLES);
    glVertex2f(0.0f, 0.0f);
    glVertex2f(1.0f, 0.0f);
    glVertex2f(0.5f, 1.0f);
glEnd();

glFlush();
}
```

# SUMMARY

- Translation → adds vector
- Rotation → uses sin/cos
- Scaling → multiplies coordinates
- Homogeneous → unifies all transformations
- Order matters in transformations O
- penGL applies transformations in reverse order